SFWRENG / MECHTRON 3K04 Software Development

This Week: Requirements Engineering

McMaster University | Engineering
Computing & Software

September 22nd, 2026

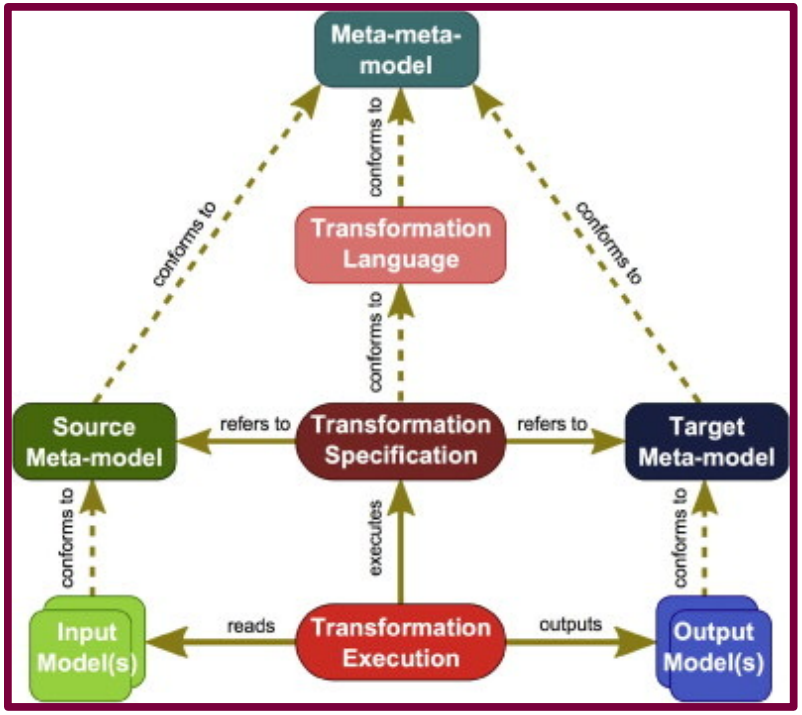
Dr. Angela Zavaleta Bernuy

Modern Trends in SDLC

- Agile:
  - Meant for smaller teams and smaller development cycles
  - Some people are distrustful of how Agile can be used to handle safety critical engineering
- Waterfall
  - Meant for bigger teams and bigger projects
  - Some people dislike how long it takes to develop using waterfall
  - Tried and tested for development of critical components
- We need a methodology that can fit in the SDLC to make development of safety critical systems efficient
  - Model-Driven Engineering!

Model-Driven Engineering (MDE)

- It makes models the main artifact of development instead of raw code
  - Code can be generated automatically from these models
  - E.g., we can transform a UML diagram into Java classes, or a state machine model into embedded system code ;)
- Goal: raise the level of abstraction so developers think in **domain concepts** rather than low-level programming
- Reduce manual coding and relies on code generation and automation tools
- **Is MDE an SDLC?**



https://www.sciencedirect.com/topics/computer-science/model-driven-engineering

MDE in Safety-critical Systems

- **Assurance**: Making statements about properties of a system and demonstrate that we have some adequate level of confidence that statements are true.
- In safety-critical systems, we want to *assure* that the system is safe, secure and dependable
- Traditionally, assurance was implicit
  - Developers would provide assurance by providing the documentation and test results of the system
- With MDE, assurance can be explicit
  - We will talk about this throughout the course

Takeaways!

- A number of SDLC models are being used and have been transformed across the years
  - Our focus is safety critical systems, for which spiral and V model are used often
  - In particular, the spiral model emphasizes the iterative nature of hazard analysis
  - Of course, we can use other SDLC models, but keep hazard/risk analysis in mind
- MDE is prevalent in many industries, specially for safety-critical systems
- Explicit assurance is needed to ensure the systems we develop are safe, secure and dependable

Now, Requirements!

Requirements & Validation

- If we don't get the requirements right, it does not matter how good we build our system.
- The world of *requirements*, can be divided into a few parts:
  - **Eliciting** requirements: how do we **capture** the requirements
  - **Documenting** requirements: how do we **keep track** of the requirements
    - **Formal specifications**: well-defined syntax and semantics (usually mathematical) – (Week 5)
    - **The 4 variable model**: relationship between requirements and design – (Week 5)
  - How do we know if we elicit and document requirements **correctly**?
- *Validation*: determining whether or not we got the *right* requirements
  - Not to be confused with *verification*, which is determining whether or not we built the system *right* (i.e., if it complies with the requirements)
  - We often refer to Validation and Verification as V & V.

Documenting Requirements: Natural Language

- Most requirements are written in natural language
- Often they consist of a succession of points or paragraphs that do not collect all related information
- Each paragraph usually contains multiple items
- What are some **problems** with using natural language?
  - It can be ambiguous, imprecise, inconsistent…
- What do we **require** from requirements?
  - For a given input, what should the output be? I.e. how should the system behave?
  - Recall the continuity property!

Infusion Pumps (2007)

Baxter COLLEAGUE Infusion Pumps Malfunction Yet Again

Oct 24, 2007 | Parker Waichman Alonso LLP

Baxter COLLEAGUE infusion pumps have been malfunctioning again under certain conditions, prompting new warnings from the company. According to a letter sent to its Canadian customers, Baxter has received reports of incidents where the COLLEAGUE triple channel infusion pumps stopped infusing. According to the "Urgent Device Correction" notice issued by Baxter, the company is working on a software solution to fix the problem with its COLLEAGUE triple channel infusion pumps.

The Baxter infusion pumps affected by the correction notice include Baxter COLLEAGUE triple channel Mono, CX and CXE Volumetric infusion pumps with product codes of 2M8153, 2M8163, 2M9163, DNM8153 and DNM9183. According to the correction notice, Baxter has received reports from 3 Canadian customers of at least six instances where defective COLLEAGUE infusion pumps stopped infusing. In each instance, the pump issued an audible and visual alarm and displayed the error code 16:310:867:0002 before it stopped. Baxter said that this malfunction of the COLLEAGUE infusion pumps occurred when the capacity of the buffer memory device was exceeded. While no injuries have been reported in relation to the defective Baxter COLLEAGUE infusion pumps, the company said that lab tests of the pumps showed that this malfunction had a high probability of occurring under certain circumstances.

Infusion Pumps (2025)

Zyno recalls infusion pumps over unvalidated software

Customers have been asked to stop using the pumps and wait for someone to contact them about exchanging devices.

Dive Brief:

- Zyno Medical has recalled infusion pumps that shipped with software that had not undergone verification and validation testing, the Food and Drug Administration said Monday.
- The company released the Z-800 pumps with incorrect software versions. The software created a risk of incorrect air-in-line detection and audio alarms that could cause serious harm.
- Zyno has not reported any serious injuries or deaths. The recall, which the FDA assigned to Class I, started eight months after Zyno removed Z-800 pumps from use over another software issue.

Who is a Stakeholder?

- Anyone who **affects or is affected by** the system, or has a say in whether it is accepted.
- Not just users. Users are a subset of stakeholders.
- Three groups:
  - **Direct users:** operate it (nurse, biomedical technician).
  - **Indirect / affected:** never touch it, still bear the outcome (patient, family).
  - **Governing / external:** set rules or accept it (Health Canada, hospital privacy officer, IT security, manufacturer).

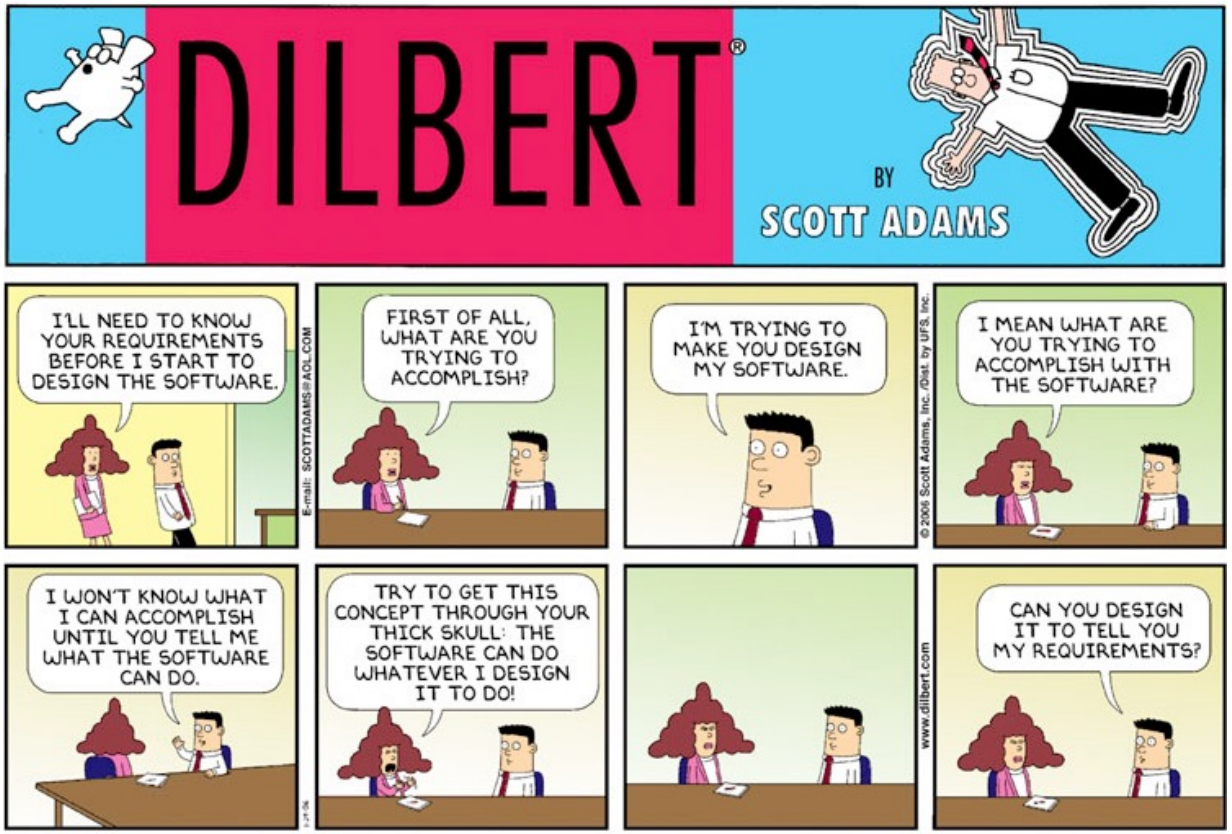
Stakeholders of the Infusion Pump

Stakeholder	Type	What they care about
Nurse	Direct user	Fast, error-proof programming; clear alarms
Patient	Affected	Correct dose, no harm, privacy
Pharmacist	Indirect user	Drug library limits are enforced
Biomedical technician	Direct user	Maintenance mode, fault logs, calibration
Charge nurse at the monitor	User of partner system	Sees alarms from every pump on the unit
Hospital IT / privacy officer	Governing	Network security; PHIPA compliance
Health Canada	Regulator	Evidence the device is safe and effective

Prioritizing Stakeholders: Power/Interest Grid

- Two axes: **power** over the project (low → high) and **interest** in it (low → high).
- High power + high interest → *manage closely* (hospital, regulator).
- Low power + high interest → *keep informed and represent* (nurses, patients).
- High power + low interest → *keep satisfied* (IT, finance).
- Low power + low interest → *monitor*.
- **Engineering twist:** in safety-critical systems, low power never means low priority. The patient has no power and the highest stake.

Requirements



© Scott Adams, Inc./Dist. by UFS, Inc.

Who? Regulatory Environments

- In safety-critical domains, you just don't build and deploy, all systems are *regulated*
- One or more government agencies are mandated to act as *regulators* and are then empowered to *licence* or *approve* systems/devices before they can be deployed
- Typical domains:
  - **Nuclear power:** Canadian Nuclear Safety Commission (CNSC), US NRC.
  - **Civil aviation:** FAA (US), Transport Canada.
  - **Railways:** Transport Canada, FRA (US).
  - **Medical devices:** Health Canada, FDA (US).
- Legal accountability: if you violate regulations (e.g., deploy an uncertified system), your organization can face fines or legal action.

Regulation vs. Standardization

- **Regulation:** mandatory, enforced by government/regulator, carries legal accountability.
- **Standards:** industry consensus on good practice, often required by regulators but not laws by themselves.
  - Typically, software standards are (too) process focused – and it is difficult to describe process in enough detail to ensure the outcome is what we need, eg safe!
  - Minimum requirement!
- Typically:
  - *Regulators say what you must achieve* (safe, secure, reliable).
  - *Standards say how you should go about it* (processes, SILs, documentation).
- Requirements engineering tells us how to write **clear specifications**.
- Regulation and standards tell us how to make sure those specifications, and the systems built from them, are **acceptable in safety-critical domains**.

Ethics in Requirements

- Requirements decide **who the system serves and who carries the risk**.
  - That is an ethical choice, whether we make it on purpose or not.
- Most harms in the Week 1 failures were built into requirements: missing limits, missing alarms, missing access control.
- Ethics cannot be tested in at the end. It is either in the requirements or it is missing.

Codes of Ethics

- **PEO Code of Ethics** (Professional Engineers Ontario, O. Reg. 941 s.77):
  - *An engineer's duty to public welfare is **paramount**, ahead of client and employer.*
- **ACM/IEEE-CS Software Engineering Code of Ethics**, 8 principles:
  - *Public, Client and Employer, Product, Judgment, Management, Profession, Colleagues, Self.*
  - Principle 1: act consistently with the public interest.
- Law for health software in Ontario:
  - **PHIPA** (personal health information)
  - **Health Canada Medical Devices Regulations**
  - **AODA** (accessibility).

Ex: From Ethical Principles to Requirements

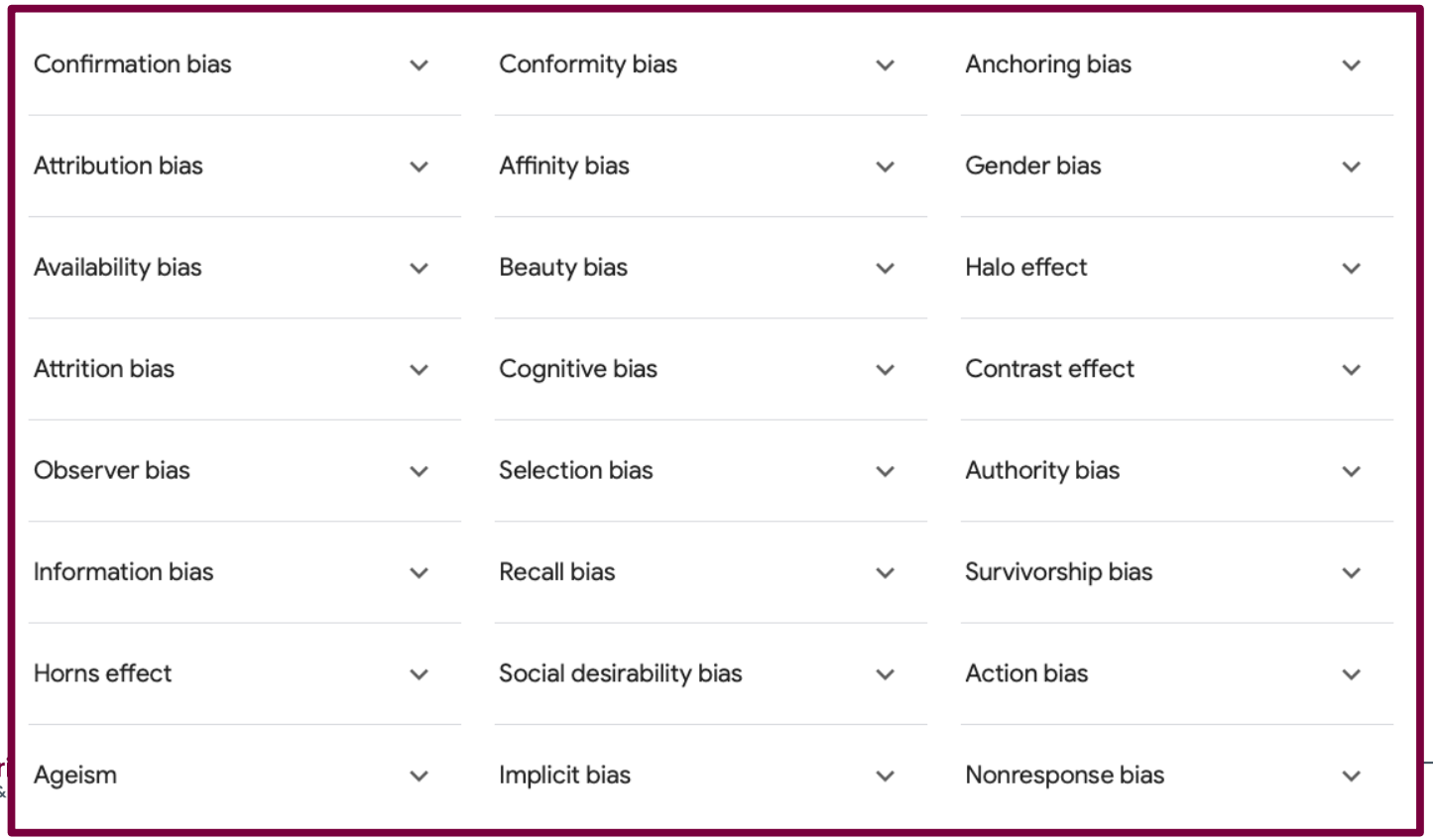
Principle	Concern in the pump	Resulting requirement (example)
Patient safety	Overdose from a typo	FR: The pump shall reject a rate above the drug library's hard limit.
Privacy	Patient name visible to anyone	NFR: Patient identifiers shall be shown only after a user logs in.
Controlled access	Anyone can change a running infusion	FR: Changing a running infusion shall require nurse credentials.
Accountability	Nobody knows who changed a dose	NFR: Every change shall be logged with user ID and timestamp.
Accessibility	Colour-only alarms	NFR: Every alarm shall have both a sound and a visual cue.
Honesty about limits	Monitor shows stale data as live	FR: The monitor shall mark data older than 30 s as "stale".

Activity: The Ethics Lens

- Pick a medicine pump feature: drug library, remote monitoring, event log, maintenance mode, etc.
  - Who could be harmed?
  - Whose data is exposed?
  - Who is left out?
- Come up with one requirement that address it, with the principle it serves.
  - E.g. *Public, Client and Employer, Product, Judgment, Management, Profession, Colleagues, Self, Accessibility, Privacy, Patient safety, etc.*

Bias in Requirements

When we google the word bias, these is what we get:



Bias in Requirements

- Biases exist everywhere
  - And these lead to assumptions!
- They are both conscious and unconscious, and help us to make sense of the requirements
  - Sometimes, poor sense of the requirements
- It is critically important when writing or evaluating requirements to be aware of biases
  - Be aware of how your bias as an engineer may inform decisions around requirements
- Diversity in opinions and experiences are extremely helpful to avoid *group think* and mitigate biases in the requirements

SFWRENG / MECHTRON 3K04 Software Development

Today: Eliciting and Documenting Requirements

McMaster University
Engineering
Computing & Software

September 24th, 2026

Dr. Angela Zavaleta Bernuy

Elicitation: Requirements are Discovered

- Stakeholders rarely hand you requirements.
  - They describe problems, habits and workarounds.
- They leave out what is obvious to them (tacit knowledge), and they disagree with each other.
- Elicitation means using several techniques and cross-checking the answers.

Requirements Elicitation Techniques

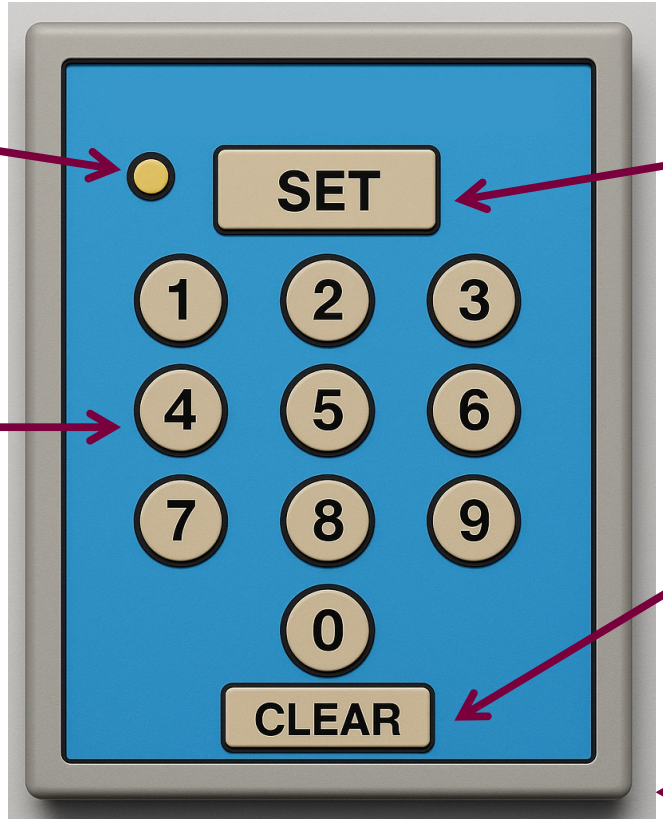
Technique	Good for	Watch out for
Interview (semi-structured)	Goals, pain points, edge cases	Leading questions; one loud voice
Observation	Tacit steps nobody mentions	Takes time; people act differently when watched
Workshop	Resolving conflicts between stakeholders	*Group think*
Document analysis	Regulations, standards, incident reports	Documents go out of date
Questionnaire	Many users, simple facts	Shallow answers
Prototype / mock-up	UI and workflow feedback	Users fixate on looks
5 Whys	Getting past a symptom to the root need	Stopping too early

Time for a larger example

Home Alarm System: Background Details

- Who is asking for an alarm system?
  - Who will use the alarm system?
  - I.e., stakeholders
- Why do we want this alarm system?
  - The 5 whys technique: https://archive.org/details/toyotaproduction0000onot
  - On average, it takes 5 whys to get to the root of a problem
- What will the alarm system do?
- Where will the alarm system go? Where will it be used?
- When will the alarm system be used?

Home Alarm System: Background Details



Light on when the system is live

Button to make system live

Buttons to enter 3-digit code to stop the alarm sound and deactivate alarm

Button to clear erroneous digit entry and re-enter all digits

Sensors are connected to the panel and will make the alarm sound when motion is detected and system is live

Home Alarm System: Requirements

1. The security system has a detector that sends a trip signal when motion is detected.
2. The security system is activated by pressing the SET button.
3. The SET button is illuminated when the system is active.
4. If a trip signal is detected while the system is active, an alarm is sounded.
5. A three-digit code must be entered to turn off the alarm sound.
6. Correct entry of the three-digit code deactivates the device.
7. If a mistake is made when entering the code, the user must press the CLEAR button before the entire code can be re-entered.

Are these requirements complete? Unambiguous? Clear?

Clear, unambiguous, understandable?

Example: specification fragment for a word-processor

> Selecting is the process of designating areas of the document that you want to work on. Most editing and formatting actions require two steps: first you select what you want to work on, such as text or graphics; then you initiate the appropriate action.

Can an area be scattered?

Consistent?

Example: specification fragment for a word-processor

> The whole text should be kept in lines of equal length. The length is specified by the user. Unless the user gives an explicit hyphenation command, a carriage return should occur only at the end of a word.

What if the length of a word exceeds the length of the line?

Precise, unambiguous, clear?

Example: specification fragment for a spacecraft (safety-critical systems)

> The message must be triplicated. The three copies must be forwarded through three different physical channels. The receiver accepts the message on the basis of a two-out-of-three voting policy.

Can a message be accepted as soon as we receive 2 out of 3 identical copies of message or do we need to wait for receipt of the 3rd?

Functional vs. Non-Functional Requirements

- **Functional (FR):** what the system *does*: a behaviour, a response to an input.
  - "*The pump shall stop the infusion when an occlusion is detected.*"
- **Non-functional (NFR):** *how well* or *under what constraints*: a quality or a limit.
  - *"The pump shall raise the occlusion alarm within 2 s of detection."*
- Quick test: if you remove it, does a feature disappear (FR) or get worse (NFR)?
- NFRs are the ones teams forget, and the ones regulators check first.

Requirements Elicitation Techniques

Category	Question it answers	Pump example
Safety	What must never happen?	*The pump shall not infuse while the door is open.*
Security	Who may do what?	*Remote commands shall be rejected unless authenticated.*
Reliability	How often may it fail?	*Fewer than 1 unplanned shutdown per 10,000 h of operation.*
Availability	When must it be usable?	*The monitor shall be available 99.9% of each month.*
Performance	How fast?	*Alarm shown at the monitor within 5 s of the pump raising it.*
Traceability / auditability	Can we reconstruct what happened?	*Every dose change logged with user ID, old value, new value and time.*
Usability	Can the intended user do it correctly?	*A trained nurse shall program an infusion in under 60 s.*

Example: Pseudocode

```
if (a=0 & b=0) then solution_caption := “Ill-conditioned”
else if (b≠0) then
   solution_caption := “One root”; x1 := -c/b
else
   d := b2-4ac
   if (d<0) then solution_caption := “No real roots”
   else if (d=0) then
            solution_caption := “One real root”; x1 = -b/(2a)
   else
            solution_caption := “Two roots”
            if (b≥0) then x1 := (-b-√d)/(2a)
            else x1 := (-b+√d)/(2a)
            x2 := c/(ax1)
```

Is this a *good* specification?
Is it easy to understand?
Is this a **requirement** or is it a **design** specification?
What is the difference?

Example: Solving a Quadratic Equation

- Is this a good specification?
  - Not really. It is too detailed. It is mixing up *what* needs to be achieved with *how* it should be done.
- Is it easy to understand?
  - Yes, for engineers, but not for non-technical people
- Is this a requirement or is it a design specification?
  - Design! It explains how to compute the solution
- What is the difference?
  - *Requirements specifications* describe required behaviour in a system (WHAT)
    - *"The calculator shall determine and report the number of real solutions of a quadratic equation and provide their values if they exist."*
  - *Design specifications* describe the behaviour prescribed in a design (HOW)
    - *"If discriminant d < 0 then return 'no real roots'. Else compute roots using x = (-b ± √d) / (2a)."*

What Makes a Good Requirement

- Based on ISO/IEC/IEEE 29148. Each requirement should be:
  - **Atomic:** one requirement per statement. No "and" joining two behaviours.
  - **Unambiguous:** only one reading. Avoid *fast, easy, appropriate, user-friendly, etc., and/or*.
  - **Verifiable:** you could design a test for it. If you cannot say how you would check it, rewrite it.
  - **Complete:** covers the off-normal cases too (recall the alarm example: what if both buttons are pressed?).
  - **Consistent:** does not contradict another requirement.
  - **Traceable:** has an ID and a source (a stakeholder, a scenario, a regulation).
  - **Implementation-free:** says WHAT, not HOW (regulators).

A Template and an ID Scheme

- Template: **[ID] The [system] shall [action] [object] [condition / constraint].**
  - "shall" = mandatory, "should" = desirable, "may" = optional. Pick one meaning and stick to it.
- ID scheme, set once and never renumbered:
  - **FR-DSP-03**: functional, dispensing capability, #3.
  - **NFR-SEC-01**: non-functional, security, #1.
  - Deleted IDs are retired, never reused.
  - Tests, assumptions, decisions logs, etc. will trace back to these IDs.
- Group FRs by capability
  - For D1: authentication, logging, fault detection, and so on.

Let's Fix These Requirements

Bad requirement	Problem	Better version
The pump should be easy to use.		
The pump shall alarm and stop and notify the monitor on occlusion.		
The system shall respond quickly to alarms.		
The pump shall use an SQLite database for logs.		
Only authorized users can change settings.		

SFWRENG / MECHTRON 3K04 Software Development

Today: User Stories and Acceptance Criteria

McMaster University | Engineering
Computing & Software

September 25th, 2026

Dr. Angela Zavaleta Bernuy

What is a User Story?

- A short description of a need, told **from a stakeholder's point of view**.
- Template: **As a ⟨role⟩, I want ⟨goal⟩ so that ⟨benefit⟩.**
- Came from Agile
  - Recall: "prioritized backlog of features or user stories"
- A requirement says what the system *shall* do. A story says *who needs it and why*.
  - D1 asks for both, linked by ID.
- A story is only done when it has **acceptance criteria**.

User Stories: What Goes Wrong

Part	Job	Weak	Strong
Role	A real stakeholder from your list	*"As a user…"*	*"As a bedside nurse…"*
Goal	One thing they want to do	*"…I want the system to work well…"*	*"…I want to pause an infusion with one action…"*
Benefit	Why it matters; the value	*"…so that I can pause it." (repeats the goal)*	*"…so that I can reposition the patient without disconnecting the line."*

Organizing Stories: Themes and Epics

- **Theme:** a group of related stories, usually one per capability.
  - *E.g. Programming, Alarms, Access control, Remote monitoring, Maintenance*.
- **Epic:** a story too big to finish or test in one go. Split it.
  - Epic: "As a nurse, I want to manage infusions from the monitor."
  - Split: view status → acknowledge an alarm → see infusion history.
- In D1, use your FR capability groups as themes, so stories and requirements line up.

INVEST: The Quality Checklist

Letter	Meaning	Ask yourself
Independent	Can be built and tested without another story	Does it need story X finished first?
Negotiable	Describes a need, not a fixed design	Does it prescribe buttons, screens or tech?
Valuable	Delivers value to a stakeholder	Would any stakeholder notice if it were missing?
Estimable	Team can size it	Do we understand it well enough to guess the effort?
Small	Fits in one iteration	Can it be demoed in a lab session?
Testable	Has pass/fail acceptance criteria	Could it be checked?

INVEST: Good or Bad?

Story	Verdict
As a nurse, I want a blue button at the bottom right so that I can pause.	
As the system, I want to store logs in a database.	
As a nurse, I want the pump to be safe.	
As a pharmacist, I want to update the drug library on every pump at once so that new dose limits apply hospital-wide the same day.	
As a charge nurse, I want to see every active pump alarm on my unit in one list so that I can send help to the right room first.	

Semi-Formal Specifications

- On one hand, we can have informal specifications (natural language), while on the other, we can have formal specifications (predicate logic, tabular expressions, data flow diagrams, etc.)
- Semi-formal specifications fall somewhere in between
- For example, the Gherkin specification languages (https://cucumber.io/docs/gherkin/reference)

Semi-Formal Specifications - Gherkin

- Gherkin combines natural language specification with some keywords to give it a more rigorous structure compared to free-hand writing
- Three main keywords are used: GIVEN, WHEN, THEN
  - **GIVEN**: used to describe the initial context of the system
    - The *scene* of the scenario. It is typically something that happened in the *past*. (Preconditions)
  - **WHEN**: used to describe an event, or an *action*.
    - This can be a person interacting with the system, or it can be an event triggered by another system. (Events)
  - **THEN**: used to describe an *expected* outcome, or *result*. (Postconditions)

Example: Semi-Formal Specifications – Gherkin

- Gherkin specification typically best used to specify scenarios

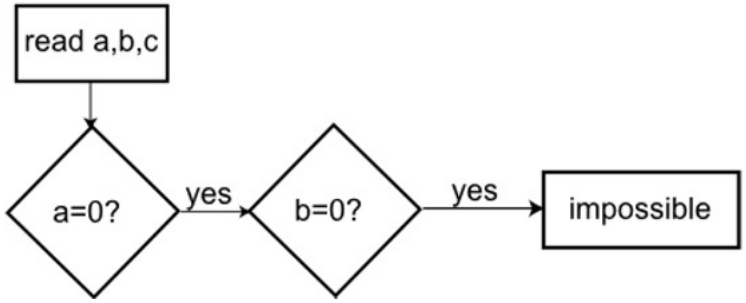
GIVEN three variables a, b, c

AND

GIVEN a = 0 and b=0

WHEN calculating the roots of a quadratic equation

THEN the resulting roots are impossible to determine



- It can also be written using **AND**: used to cojoin multiple **GIVEN/WHEN/THEN** statements

Example: Semi-Formal Specifications – Gherkin

- Can also be written using **BUT**: used to conjoin multiple **GIVEN/WHEN/THEN** statements, usually a constraint or an opposing specification

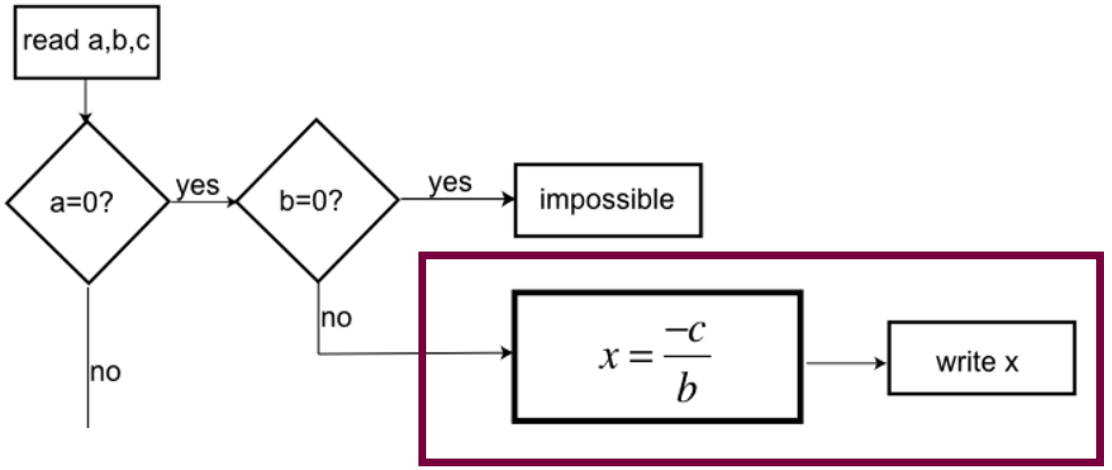
GIVEN three variables a, b, c

AND a = 0

BUT b ≠ 0

WHEN calculating the roots of a quadratic equation

THEN the resulting root x = -c/b



Acceptance Criteria: When is the Story Done?

- Specific, pass/fail conditions that a tester can check **without asking the team**.
- Written in Gherkin:
  - **Given** context, **When** event, **Then** observable outcome.
- Aim for 3–5 per story: **1 normal path + at least 1 off-normal path** (fault, invalid input, lost connection).
- Every "Then" must be observable: a display, a sound, a log entry, a message. Not "the system handles it".
- Numbers from your NFRs belong here (timeouts, limits).

Example

US-ALM-02 (traces to FR-ALM-02, FR-COM-04, NFR-PRF-02): As a **charge nurse at the Nurse Station Monitor**, I want to **see an occlusion alarm from any pump on my unit** so that **I can send a nurse before the patient misses medication**.

Scenario 1: Alarm reaches the monitor (normal)
GIVEN pump P-07 is infusing and connected to the monitor
WHEN P-07 detects an occlusion
THEN the monitor shows "Occlusion – Room 412 – P-07" within 5 s
AND the alarm is logged with pump ID and time

Scenario 2: Connection lost (off-normal)
GIVEN pump P-07 is infusing
AND the monitor has received no message from P-07 for 30 s
WHEN the monitor checks P-07's status
THEN the monitor shows "P-07 – communication lost" in the alarm list
BUT the pump keeps its own local alarm sounding

Scenario 3: Acknowledgement
GIVEN an occlusion alarm for P-07 is shown
WHEN the charge nurse acknowledges it
THEN the alarm is marked acknowledged with the nurse's user ID
AND it stays in the list until the pump reports it cleared

Acceptance Criteria: Exercise

- Given this requirement: FR-AUT-03: **The pump shall require a nurse or technician login before any infusion parameter can be changed.**
- Write a user story: real goal and real benefit
- Write 3 acceptance criteria: 1 normal, 1 invalid input, 1 fault
- Check them against INVEST.

Traceability

- It answers: *why does this requirement exist, where is it built, and how do we know it works?*
- **Backward:** requirement → the stakeholder or regulation it came from.
- **Forward:** requirement → story → component → module → test.
- Regulators ask for it (recall: every document traces to the next).
- It makes change safe: when a stakeholder changes their mind, the table shows everything affected.

Traceability Table

Req ID	Requirement (short)	Story	Component (next week!)	Stakeholder / source
FR-PRG-04	Reject rate above hard limit	US-PRG-02	Dose Checker	Pharmacist; drug library
FR-AUT-03	Login before changing infusion	US-AUT-01	Access Control	Nurse; privacy officer
FR-ALM-02	Alarm on occlusion	US-ALM-01, US-ALM-02	Pump Controller, Alarm Manager	Nurse; Health Canada recall notices
NFR-PRF-02	Alarm at monitor ≤ 5 s	US-ALM-02	Comms Link, Monitor Alarm List	Charge nurse
NFR-AUD-01	Log every dose change	US-AUD-01	Event Logger	Hospital risk management

Traceability Table Check

- **Orphan requirement:** no stakeholder or source
  - Why is it here? Delete it or find the source.
- **Orphan story:** no requirement
  - A requirement is missing. Write it.
- **Requirement with no story:** usually fine for internal NFRs.
  - Say so explicitly.
- **Component with no requirement:** gold-plating, or a missing requirement.
- **One requirement in 6 components:** probably not atomic.
  - Split it.

Questions?